INF1022 - Trabalho Final: OrqLang

Disciplina: Analisadores Léxicos e Sintáticos | Semestre: 2026.2

Professor: Vitor Pinheiro de Almeida | Modalidade: individual ou dupla

Entrega: data e horário a serem informados no EAD.

1. Objetivo

Desenvolver o front-end de um compilador para a linguagem OrqLang, uma DSL de orquestração de tarefas e serviços. O programa deve realizar análise léxica e sintática e transpilar um arquivo .orq para um programa Python executável. Os serviços Python serão fornecidos pelo professor. O foco da avaliação é a implementação do analisador e da tradução, não a implementação dos serviços.

É permitido utilizar PLY (Python), Flex/Bison (Lex/Yacc) ou ferramenta equivalente baseada em gramática. Não é permitido substituir o analisador por uma coleção de expressões regulares que reconheça diretamente os programas completos.

2. Ambiente e funcionamento

O professor fornece o pacote orqlang-kit contendo runtime/servicos.py, runtime/orqlang_runtime.py (bônus), documentação, exemplos e testes. O transpilador deve aceitar um arquivo .orq e produzir um arquivo .py. O programa Python gerado deve usar as funções de runtime.servicos. Não há necessidade de conhecer APIs HTTP, nuvem ou bancos de dados.

```
arquivo.orq  -->  [lexer + parser + gerador]  -->  arquivo.py
                                                       |
                                              runtime.servicos
```

3. Gramática inicial obrigatória

A notação a seguir é EBNF: { X } significa zero ou mais ocorrências e [X] indica item opcional. Os terminais em maiúsculas são palavras reservadas, salvo ID, NUM e STR. Espaços e quebras de linha separam tokens, mas não delimitam comandos; a gramática deve resolver a estrutura dos blocos.

```
programa   ::= INICIO comandos FIM
comandos   ::= comando { comando }
comando    ::= VAR ID = expr
             | SERVICO ID ( [ argumentos ] ) [ -> ID ]
             | IMPRIMIR expr
argumentos ::= expr { , expr }
expr       ::= ID | NUM | STR | VERDADEIRO | FALSO
```

A gramática inicial aceita literais inteiros e decimais em NUM (por exemplo 10 e 2.5), strings entre aspas duplas, identificadores iniciados por letra ou sublinhado, e booleanos VERDADEIRO/FALSO. Comentários iniciados por # até o fim da linha são ignorados. Palavras reservadas são sensíveis a maiúsculas/minúsculas e devem ser escritas como mostrado.

4. Extensões obrigatórias da gramática

A. Expressões: adicionar os operadores aritméticos +, -, *, /; comparações ==, !=, <, <=, >, >=; e parênteses. Da menor para a maior precedência: (1) comparações; (2) soma e subtração; (3) multiplicação e divisão; (4) sinal negativo unário; (5) valores, identificadores e expressões entre parênteses. Operadores binários aritméticos têm associatividade à esquerda. Não são aceitas comparações encadeadas como 1 < x < 10. A divisão / segue Python: 5 / 2 resulta em 2.5. Comparações produzem booleanos.

Tabela de precedência dos operadores

Ordem	Operadores
1 — Menor	Comparações: ==, !=, <, >, <=, >=
2	Soma e subtração: +, -
3	Multiplicação e divisão: *, /
4	Sinal negativo: -5, -x
5 — Maior	Valores, variáveis e parênteses

O que é menos unário? É o sinal - aplicado a um único valor ou expressão para inverter seu sinal, por exemplo -5, -preco ou -(2 + 3). É diferente da subtração, que tem dois operandos: 10 - 3. Exemplo: -2 * 3 é interpretado como (-2) * 3.

O que são átomos? São os elementos mais simples de uma expressão: números (10, 2.5), strings ("Ana"), booleanos (VERDADEIRO, FALSO) e variáveis (preco). Uma expressão entre parênteses, como (2 + 3), também pode ser usada como unidade na expressão. Não é necessário criar um token chamado ATOMO: esse é apenas um nome comum para uma categoria de regras gramaticais.

Extensão opcional: operadores lógicos E, OU e NAO. Se implementados, devem ser documentadas sua precedência e tradução para and, or e not em Python. Sugestão: OU < E < NAO < comparações, com NAO aplicado a uma condição completa.

B. Condicional: SE expr ENTAO comandos [SENAO comandos] FIM. Blocos podem ser aninhados. O SENAO pertence ao SE mais próximo ainda aberto. A condição segue a regra de verdade do Python.

C. Repetição: REPITA expr VEZES comandos FIM. A expressão de contagem é avaliada uma única vez na entrada do bloco, deve produzir inteiro não negativo e controla exatamente essa quantidade de iterações. Não é exigida validação estática do tipo da contagem.

D. Reatribuição: VAR ID = expr também pode atualizar uma variável já existente. Não é exigida tabela de símbolos nem declaração prévia.

5. Extensão opcional: paralelismo (bônus)

Adicionar PARALELO chamadas FIM, em que chamadas são duas ou mais produções SERVICO ID ([argumentos]) -> ID. Qualquer função da biblioteca pode ser chamada, mas dentro do bloco são permitidos somente comandos SERVICO com variável de saída; não são permitidos VAR, IMPRIMIR, SE, REPITA ou PARALELO aninhado. Cada serviço executa independentemente dos demais. Após todos terminarem, a execução continua depois do FIM.

Variáveis de saída distintas: cada chamada dentro do mesmo bloco PARALELO deve armazenar seu resultado em uma variável diferente. Não é permitido que duas ou mais chamadas atribuam seus resultados à mesma variável.

Exemplo inválido: as duas chamadas usam a mesma variável resultado:

```
PARALELO
    SERVICO buscar_preco(10) -> resultado
    SERVICO verificar_estoque(10) -> resultado
FIM
```

Exemplo válido: cada serviço utiliza uma variável de saída diferente:

```
PARALELO
    SERVICO buscar_preco(10) -> preco
    SERVICO verificar_estoque(10) -> estoque
FIM
```

```
IMPRIMIR preco
IMPRIMIR estoque
```

Ao final do bloco válido, preco e estoque estarão disponíveis para os comandos seguintes. Os argumentos de todas as chamadas são avaliados antes de iniciar a execução paralela e não podem depender de resultados produzidos por outras chamadas do mesmo bloco. O runtime auxiliar fornecido implementa a concorrência; os alunos apenas geram o código que o utiliza.

6. Tradução e comportamento esperado

Chamadas de serviço podem ter ou não variável de saída, e podem receber zero ou mais argumentos. Ex.: SERVICO registrar_log("Inicio") gera registrar_log("Inicio"); SERVICO buscar_usuario(1) -> nome gera nome = buscar_usuario(1). O valor retornado por um serviço sem -> é simplesmente ignorado.

O Python gerado deve preservar a ordem de execução sequencial, produzir saídas via print para IMPRIMIR, gerar if/else e laços equivalentes, e chamar serviços pelo nome indicado no programa. Exemplo: SERVICO buscar_usuario(1) -> nome gera uma atribuição equivalente a nome = buscar_usuario(1).

Exemplos de expressões: 2 + 3 * 4 vale 14; (2 + 3) * 4 vale 20; 10 - 3 - 2 vale 5; -5 + 8 vale 3. Comparações não podem ser encadeadas. A condição de SE utiliza a regra de verdade de Python.

O gerador pode usar código Python com indentação correta, AST própria ou tradução dirigida pela sintaxe. O arquivo gerado deve importar from runtime.servicos import * e ser executado com a pasta raiz do kit no PYTHONPATH. Não é necessário validar existência, quantidade ou tipos de argumentos de serviços; erros de execução da biblioteca podem ser propagados.

7. Exemplo completo

```
INICIO
VAR usuario = 1
VAR quantidade = 2
SERVICO buscar_usuario(usuario) -> nome
SERVICO buscar_preco(10) -> preco
SERVICO calcular_total(preco, quantidade) -> total
SE total >= 150 ENTAO
    SERVICO aplicar_desconto(total, 10) -> final
    IMPRIMIR final
SENAO
    IMPRIMIR total
FIM
FIM
```

Saída esperada: 180.0. Um exemplo de Python equivalente é:

```
from runtime.servicos import *
usuario = 1
quantidade = 2
nome = buscar_usuario(usuario)
preco = buscar_preco(10)
total = calcular_total(preco, quantidade)
if total >= 150:
    final = aplicar_desconto(total, 10)
    print(final)
else:
    print(total)
```

8. Testes e tratamento de erros

O transpilador deve aceitar todos os programas válidos fornecidos em exemplos/ (exceto o exemplo paralelo, se o bônus não for implementado), produzir Python sintaticamente válido e gerar a saída indicada nos arquivos testes/*.esperado.txt. Deve também rejeitar programas malformados com

mensagem de erro léxico ou sintático e, quando possível, linha e token associados.

Exemplos de erros a rejeitar: falta de FIM; ausência de ENTAO; chamada de serviço sem fechamento de parênteses; caracteres ilegais; operador sem operando. Não é necessário implementar recuperação de erros ou diagnóstico semântico.

9. Entregáveis

Entregar um único arquivo ZIP contendo: (i) código-fonte do lexer, parser e gerador; (ii) instruções de instalação e execução; (iii) ao menos cinco programas .orq de teste, incluindo um inválido; (iv) saídas esperadas e/ou resultados; (v) relatório curto com gramática final, decisões de tradução, testes e limitações. Identificar nomes e matrículas dos integrantes.

10. Critérios de avaliação

Item	Peso
Análise léxica (tokens, comentários, erros)	15%
Análise sintática e extensões obrigatórias	30%
Geração correta de Python	25%
Execução dos testes e rejeição de inválidos	20%
Relatório, gramática e instruções	10%

Bônus: até 10 pontos percentuais adicionais pela implementação correta de PARALELO, sem ultrapassar a nota máxima definida pela disciplina. A entrevista/demonstração poderá incluir programas inéditos e alterações simples nos exemplos.

11. Orientações

Comece pequeno: reconheça o programa e as chamadas simples; depois implemente expressões, blocos, condicionais e repetição. O gerador não deve conter regras específicas para nomes como buscar_usuario: qualquer ID válido deve poder representar uma função Python.

Observação: o PDF é o enunciado. O pacote orqlang-kit fornece a biblioteca e os exemplos, mas não contém a implementação do transpilador.